Evolution of a Haskell Repository and its Use of Monads: An Exploratory Study of Stackage

Paul Leger
Universidad Católica del Norte
Coquimbo, Chile
pleger@ucn.cl

Felipe Ruiz
Universidad Católica del Norte
Coquimbo, Chile
felipe.ruiz@alumnos.ucn.cl

Nicolás Sepúlveda
Universidad Católica del Norte
Coquimbo, Chile
nicolas.sepulveda@alumnos.ucn.cl

Ismael Figueroa
Pragmatics Lab
Coquimbo, Chile
ifigueroap@gmail.com

ABSTRACT

Package repositories for a programming language are increasingly common. A repository can keep a register of the evolution of its packages. In the programming language Haskell, with its defining characteristic monads, we can find the Stackage repository, which is a curated repository for stable Haskell packages in the Hackage repository. Despite the widespread use of Stackage in its industrial target, we are not aware of much empirical research about how this repository has evolved, including the use of monads. This paper presents an empirical study that covers the evolution of fourteen Long-Term Support (LTS) releases (period 2014 - 2020) of available packages (12.46 gigabytes), including the use of monads from the mtl package that provides the standard monad core (*e.g.,* state, reader, continuations). To the best of our knowledge, this is the first large-scale analysis of the evolution of the Stackage repository with regard to packages used and monads. Our findings show, for example, a growing trend of packages is depending on other packages whose versions are not available in a particular release of Stackage; opening a potential stability issue. Like previous studies, these results may evidence how developers use Haskell and give guidelines to Stackage maintainers.

ACM Reference Format:
Paul Leger, Felipe Ruiz, Nicolás Sepúlveda, and Ismael Figueroa. 2022. Evolution of a Haskell Repository and its Use of Monads: An Exploratory Study of Stackage. In *The 37th ACM/SIGAPP Symposium on Applied Computing (SAC '22), April 25–29, 2022, Virtual Event,* . ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3477314.3506982

1 INTRODUCTION

One of the most known *pure* functional programming languages is Haskell [14] because of explicitly representing *side-effects* in the type system (*e.g.,* input/output expressions, mutable state, exceptions). To allow developers to have the benefits of side-effects, Haskell provides the *monad* abstraction [23, 32, 33], a denotational approach to embed and reason about *notions of computation* such as mutable state, I/O, exceptions, etc. Although monads can be described as a "design pattern" [11], monads additionally have strong theoretical foundations to represent side-effects. Haskell developers can use Stackage [30], a repository of curated Haskell packages that come from a central package repository named Hackage [13]. By 2020, Stackage community has released fourteen Long-Term Support (LTS) releases, starting from 2014. Additionally, Haskell developers have downloaded around 71 million of packages from the Stackage selection to date (September 2021).

To the best of our knowledge, although Stackage is the selection of the main repository for Haskell packages, we largely find only studies that focus on the use of abstractions in Hackage [9, 10, 24] and none on how the packages repository has evolved, including the use of monads. As this is a vague question, this paper focuses on several quantitative results obtained from an empirical investigation about the evolution of the Stackage repository. As Stackage is a package selection of Hackage, this repository uses the *Cabal* system [15], which we can use as a tool to directly download, build, and install packages from Hackage.

Using the Cabal system and parsing source code, we can collect information as description and package dependencies to carry out this study. This information allows us to perform an exploratory and quantitative study that consists in a massive analysis of packages of fourteen releases (12.46 gigabytes). In addition to describing the Stackage evolution, for each release we answer the following Research Questions (RQs):

- **RQ1**. What packages are the most imported by other packages in Stackage? Do packages have *unstable* or *incompatible* dependencies according to its Stackage release?
- **RQ2**. What is the average of (in)direct dependencies per package?
- **RQ3**. How frequently are the packages updated?

Regarding monads, this study focuses on a package that is widespread used: mtl (monad transformation library) package [18]. We analyze the evolution of dependencies of monad modules available in this package:

- **RQ4**. How has the use of monads evolved?
- **RQ5**. How many packages that depend on the mtl package are added and removed in Stackage? How many packages

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SAC '22, April 25–29, 2022, Virtual Event,
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-8713-2/22/04…$15.00
https://doi.org/10.1145/3477314.3506982

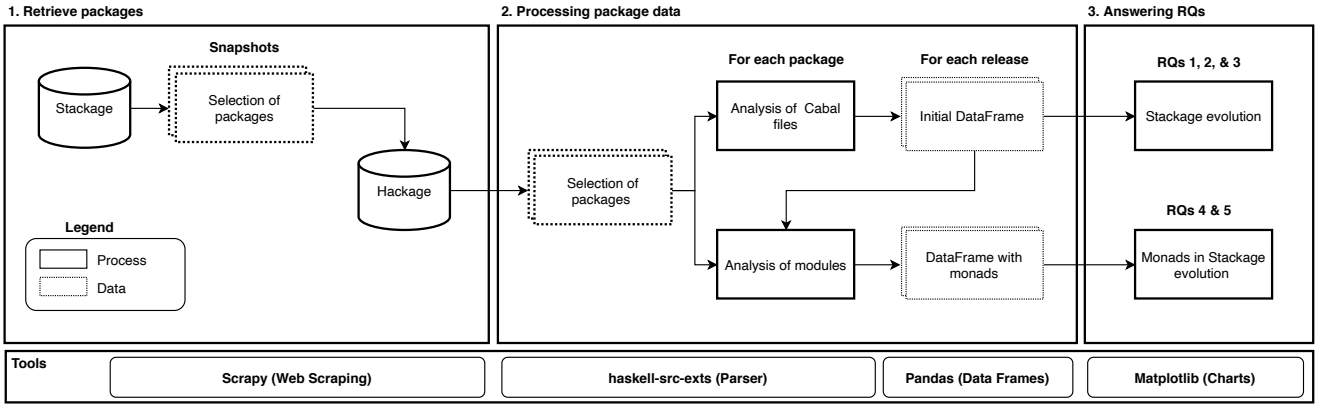


Figure 1: Workflow of our methodology

that depended on the mtl package stopped their dependencies?

Answering previous questions, we hope to provide language researchers and Stackage administrators with empirical information for the design and development of policies for language repositories. From this research result, we can provide some insightful examples. First, a growing trend of packages are *a*) depending on other packages whose versions are not available in its particular release and *b*) having dependencies with incompatible versions. Second, the mtl package is the sixth most used package in Stackage. Third, although mtl is widely used, there is *not* a growing trend to its use.

The rest of this paper is organized as follows. The next section briefly presents concepts of the evolution of package repositories and monads. Section 3 introduces the methodology used for this study. Section 4 describes the Stackage evolution through different graphs. We then reply and discuss research questions in Section 5. This paper concludes with future work and limitations of this study.

Availability. Scripts used to download and process data on https://github.com/pragmaticslaboratory/stackage-evolution (revision 8a96e75). Additionally, downloaded data are available on the same URL.

2 EVOLUTION OF PACKAGE REPOSITORY AND MONADS

This section presents the core concepts that are necessary to understand this study. First, we introduce concepts of package repositories, Mining Software Repositories (MSR) [16], and repository evolution. As two RQs are related to monads, we briefly introduce monads and their uses.

2.1 Package Repository Evolution

A package repository allows developers to store and access to packages (*e.g.,* GitHub[1]). Some repositories are specialized for one programming language. Using specialized repositories, a developer community of a language can create, access, and share packages. As examples, we can find NPM [25] for JavaScript, PYP [28] for Python, and Hackage [13] for Haskell.

A repository can evolve with different releases over time, where each release contains a list of selected packages with specific versions and does not potentially have conflict dependency among them. In Haskell, developers can use Stackage [30], a repository of curated Haskell packages that are selected from Hackage. Starting 2014, Stackage keeps a set of Long-Term Support (LTS) releases. Apart from repositories for programming languages, we can find package repositories with evolution in other solutions like Linux distributions (*e.g.,* Debian[2]).

In MSR, a field that analyzes the data available in software repositories, empirical studies have allowed developers and researchers to know a general panorama of a repository [6–8, 27]. For these studies, metrics and tools are used to analyze in detail how repositories evolve and are distributed. For example, the authors in [27] conduct a study about the evolution of a repository for the R programming language using the package metadata. Another example is in [7], where the authors analyze the NPM repository to identify the *technical lag* [12] in dependencies, *i.e.,* outdated dependencies.

2.2 Monads in Haskell

In Haskell, monads are abstractions that allow developers to encapsulate and reason *notions of computations* such as mutable state, I/O, or exceptions through a denotational approach. These abstractions was first proposed by Moggi [22] and Walder [32]. Then, Liang *et al.* [20] extended the concept to monad transformers. Additionally, monads have theoretical foundations [22, 23] that have even been applied in other areas like Aspect-Oriented Programing [5]. A library that standardizes the use of flexibly composable monads is *monad transformer library* (*a.k.a.* mtl) [18]. Other alternatives monad libraries such as transformers, monads-tf, or monads-fd with custom implementations exist, but this study focuses on mtl mainly because of its widespread usage.

There are studies like found in [1, 2, 17] regarding the use of monads in Haskell. In [1], the paper shows how monads can also be used to statically track data dependencies. The authors in [2] explain the impact of monads in terms of applications in pure functional languages like Haskell. Finally in [17], the paper presents

[1] http://github.com.

[2] http://www.debian.org/distrib/packages.

the monatron monad to address flexibility and expressiveness issues in existing monads.

3 METHODOLOGY

Following guidelines of MSR methodologies [16] and similar studies [9, 10], we defined the methodology for our study. As shown in Figure 1, this methodology is composed of three stages: retrieving packages, processing package data, and answering research questions. In each stage, we used different tools of Python. We used the *Scrapy* [29] tool to download packages, whose links to download are available on the Stackage Web. To carry out the analysis of packages, we used the *Pandas* [26] and *Matplotlib* [21] packages. Finally, we used the *haskell-src-exts* (HSE) package from Haskell to parse package modules. We next describe each stage of our methodology.

3.1 Retrieving Packages

Our study considers all releases published by Stackage between 2014 and 2020. In this period of time, this repository has published fourteen releases, where each one contains around 2,000 packages (more information in Section 4).

Using the Scrapy tool, we automatically downloaded Haskell packages. Scrapy provides *Spiders* that allow developers to define how to extract information from a Website. We started retrieving the package list of a specific Stackage release. As Stackage is a package selection of Hackage, Scrapy directly downloads packages from Hackage. This process took one hour per release.

3.2 Processing Package Data

To process the package data, we used DataFrames in Pandas because of its flexibility to (re)structure data. In each package, we can find a .Cabal file that contains its manifest. Using Haskell, we implemented a script to statically analyze these files and obtain meta-data such as their dependencies, categories, and the provided modules. To clarify the meta-data processed, we briefly describe the fields that allowed us to answer the research questions:

- **Release:** A unique string in Stackage, which indicates the release version.
- **Name:** A unique string in Hackage. We can find the same package in different releases.
- **Version:** A string that follows the format X.Y.Z (Major.Minor. Patch). We can find the same package with different versions in the releases.
- **Stability:** Free string without a format, which can be added by the developer. This field reports the stability of the package.
- **Category:** A list of free strings that describe the categories to which the package belongs. This list must be defined by the developer.
- **Dependencies:** A list of package names and their version ranges.
- **Provided modules:** A list of modules that are publicly exposed by the package.
- **Main modules:** A list of executable modules in the package. A package can specify multiple executables each with a main module.

With the retrieved meta-data, we inspected the dependencies of each package to quantify how the mtl package [18] is required. To achieve this, we used another Haskell script with HSE to allow parse the action to be performed. In this step, we added new fields to our DataFrames that describe the mtl modules used by each package. With this, we can label and count the use of each monad. In detail, the new fields added are:

- **Imported modules:** A set of package names that are imported into the source code of each package. Each module appears only once, even if it is imported into other files.
- ***mtl-direct flag*:** This flag indicates whether or not the package depends on the mtl package, that is that mtl appears in its dependencies field.
- ***Cont flag*:** This flag indicates whether or not the package modules import at least one of the following modules, which are related to the continuation monad in mtl:
  - Control.Monad.Cont
  - Control.Monad.Cont.Class
- ***Error flag*:** This flag indicates whether or not the modules in the package import at least one of the following modules, which are related to the error monad in mtl:
  - Control.Monad.Error
  - Control.Monad.Error.Class
- ***Identity flag*:** This flag indicates whether or not the modules of the package import the module Control.Monad.Identity, which is related to the identity monad in mtl.
- ***List flag*:** This flag indicates whether or not the modules of the package import the module Control.Monad.List, which is related to the list monad in mtl.
- ***RWS flag*:** This flag indicates whether or not the package modules import at least one of the following modules, which are related to the RWS monad in mtl:
  - Control.Monad.RWS
  - Control.Monad.RWS.Class
  - Control.Monad.RWS.Lazy
  - Control.Monad.RWS.Strict
- ***Reader flag*:** This indicates if the modules of the package import at least one of the following modules, which are related to the read monad in mtl:
  - Control.Monad.Reader
  - Control.Monad.Reader.Class
- ***Writer flag*:** This flag indicates whether the package modules import at least one of the following modules, which are related to the write monad:
  - Control.Monad.Writer
  - Control.Monad.Writer.Class
  - Control.Monad.Writer.Lazy
  - Control.Monad.Writer.Strict
- ***State flag*:** This indicates whether or not the modules of the package import at least one of the following modules, which are related to the state monad in mtl:
  - Control.Monad.State
  - Control.Monad.State.Class
  - Control.Monad.State.Lazy
  - Control.Monad.State.Strict

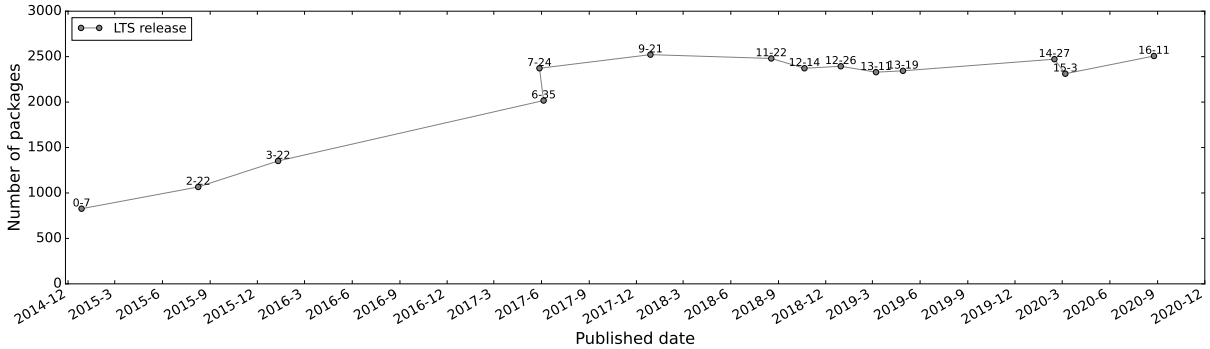
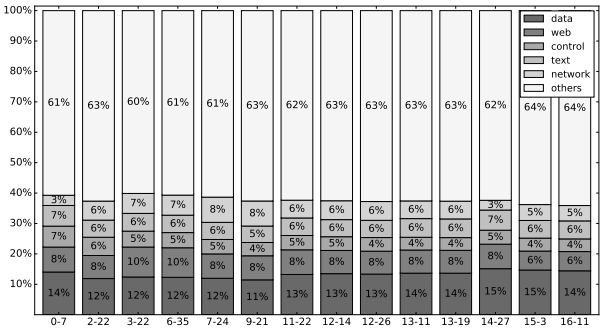
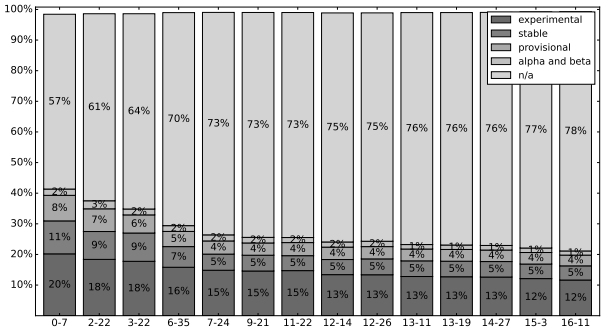


Figure 2: Stackage releases

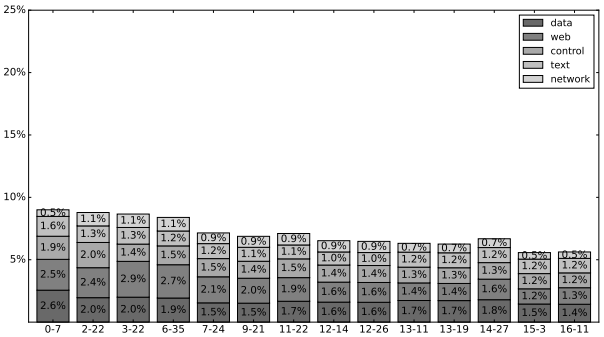


(a) Package percentage by category

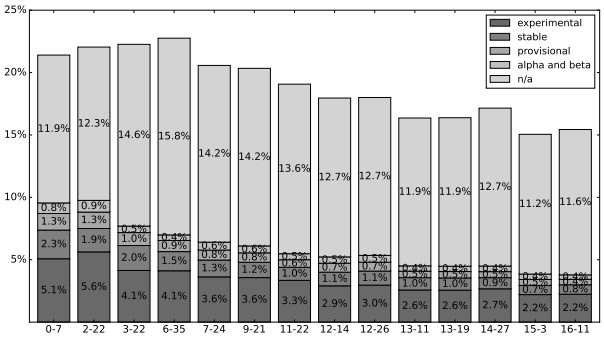


(b) Package percentage by stability level

Figure 3: Empirical distribution of the percentage of packages with respect to category and stability level



(a) Package percentage of mtl by category



(b) Package percentage of mtl by stability

Figure 4: Empirical distribution of the percentage of mtl packages with respect to category and stability level

- ***Trans flag*:** This indicates whether or not the modules of the package import at least one of the following modules, which are related to the transformer monad in mtl:
  - Control.Monad.Trans
  - Control.Monad.Trans.Class
- ***Except flag*:** this flag indicates if the modules of the package import the module Control.Monad.Except, which is related to the exception monad in the mtl, available only since version 2.2.1

3.3 Answering Research Questions

This stage goal is to answer the five research questions of this study. To answer these questions, the downloaded data was structured and processed in DataFrames using Panda. We briefly describe the process carried out for each question using these data frames.

3.3.1 RQ1: What packages are the most imported by other packages in Stackage? Do packages have unstable or incompatible dependencies according to its Stackage release? Using the *dependencies* field,

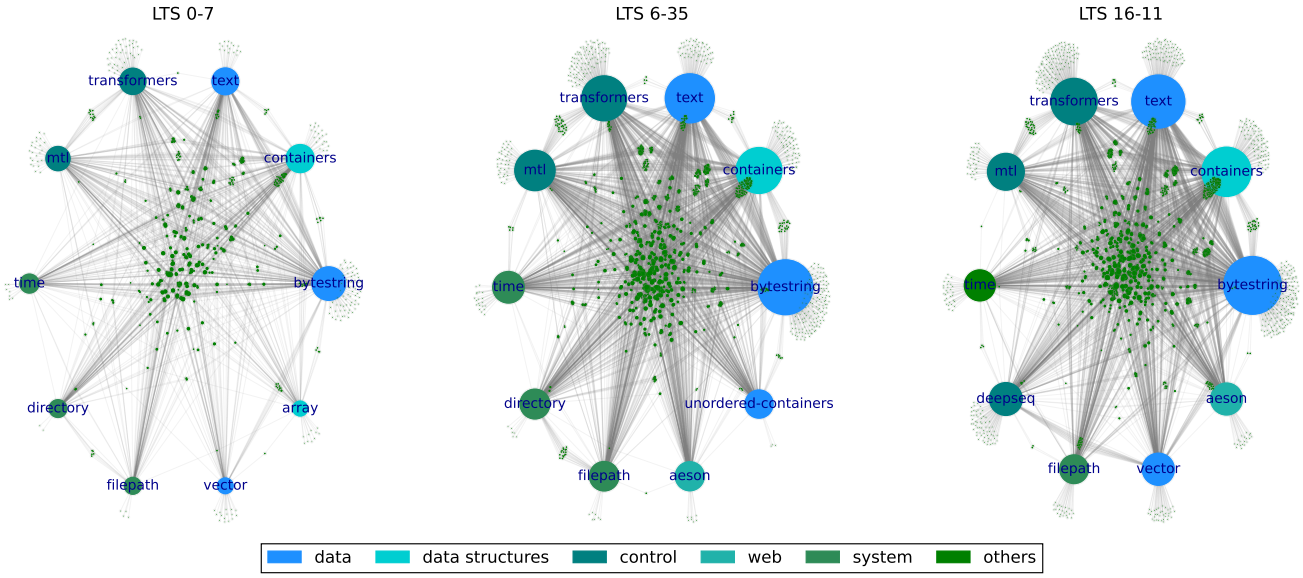


Figure 5: Packages Dependencies

we created a list of dependencies (with their versions) for each package in a release. According to the dependencies, we created a dependency network graph, where it is possible to observe the top 10 packages most used/imported in some releases of the Stackage evolution. Additionally, we found out the (number of) packages that depend on versions of packages that are not available in its Stackage release. Using the version information per package, we discovered incompatible dependency versions per release, *i.e.,* at least two packages that depend on the same package with no intersections of ranges of versions.

3.3.2 RQ2: What is the average of (in)direct dependencies per package? We carried out a *Depth First Search (DFS)* [31] in package dependencies for each release used in this study. This DFS allowed us to quantify the average of (in)direct dependencies per package. Additionally, this analysis allowed us to determine how representative are these averages.

3.3.3 RQ3: How frequently are the packages updated? We counted the number of packages that updated their versions for each release. We then compared the number of *updated* packages to the number of all packages in a release. Using this result, we can have a proxy for *technical lag* [12] in dependencies, which is useful to know how frequently bug fixes and new features are included in packages.

3.3.4 RQ4: How has the use of monads evolved? For this question, we parsed source code of each package to know which packages import mtl modules (listed in Section 3.2). Then, we compared the number of packages that import these modules to the number of all packages in a release.

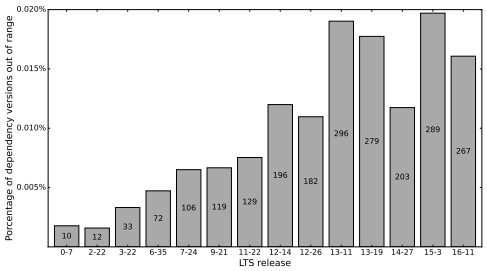
3.3.5 RQ5: How many packages that depend on the mtl package are added and removed in Stackage? How many packages that depended on the mtl package stopped their dependencies? For the first part of this question, we counted each package added and removed that imported mtl modules in a release. For the second part, we followed a similar process where it differs because we counted packages that used mtl modules and then stopped using them.

4 STACKAGE

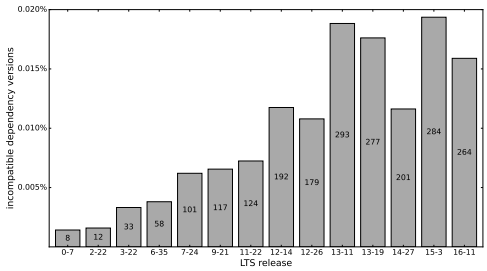
Stackage [30] is a repository of curated Haskell packages that come from a central package repository named Hackage [13]. Between 2014 and 2020, Stackage community has released fourteen Long-Term Support (LTS) releases. Before answering research questions of this paper, we briefly describe this repository using five graphs.

Figure 2 shows the release evolution during the time, specifically showing the number of packages per release and since when these releases are available. From the figure, we can see how Stackage had a gradual growth of packages throughout its first releases, highlighting that there is an interval of one or more years among releases until the release 6-35. From the release 7-24 onwards, the number of packages did not undergo major changes when compared to the first releases. Finally, there are updates that are made in short intervals of time. For example, we can see that since the release 11-22, there are 5 updates in approximately one year.

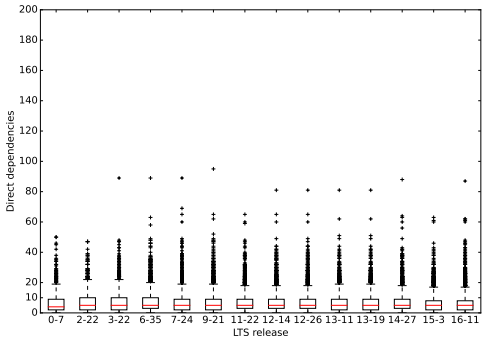
Figure 3 describes the release evolution through categories and stabilities of the packages. Figure 3a presents how the top five categories (*i.e.,* data, network, control, text, web) evolve over different releases. From the figure, we can see how some categories decreased their percentage of packages from the first release; for example, the web and control categories that had a decrease of 2% and 3% respectively. In addition, others such as data that, despite varying over time, data average percentage remained constant. Finally, we have the network category, which increases its presence in packages over time. Regarding Figure 3b, we can observe that throughout its releases, the information in the stability field has been decreasing by a notable percentage, resulting in the largest number of packages not having a defined type of stability. As an observation, the most



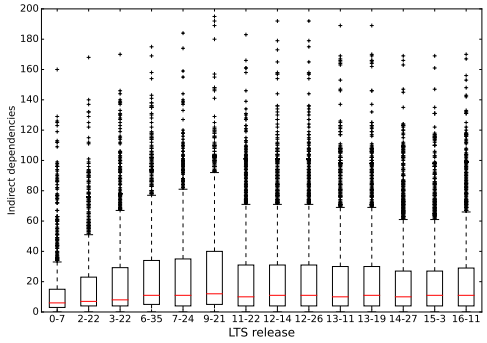
(a) RQ1 (part 2-inestable): Do packages have *inestable* dependencies according to its Stackage release?



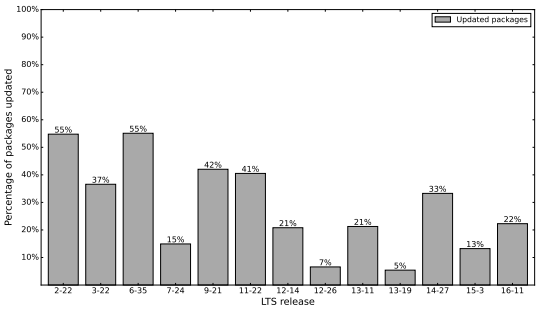
(b) RQ1 (part 2-incompatible): Do packages have *incompatible* dependencies according to its Stackage release?



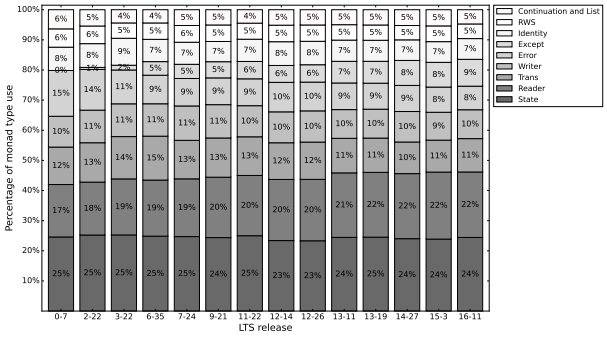
(c) RQ2 (part 1): What is the average of direct dependencies per package?



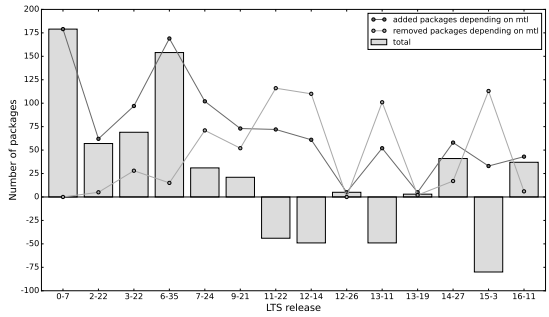
(d) RQ2 (part 2): What is the average of indirect dependencies per package?



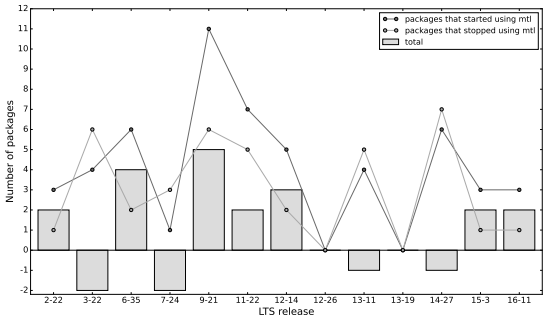
(e) RQ3: How frequently are the packages updated?



(f) RQ4: How has the use of monads evolved?



(g) RQ5 (part 1): How many packages that depend on the mtl package are added and removed in Stackage?



(h) RQ5 (part 2): How many packages that depended on the mtl package stopped their dependencies?

Figure 6: Charts to answer the research questions

of packages with information in the stability field are classified as experimental.

For the use of the mtl package in Stackage, Figure 4a shows the percentage of packages of the top five categories that use mtl considering all packages in a specific release. The figure shows that the top five categories have a clear degree of influence, despite being low percentages compared to Figure 4a. This is because with the total number of packages in each category, there is a visible percentage belonging to packages that use mtl. On the hand one, Figure 4b shows how the packages that have defined stability were decreasing the use of mtl. On the other hand, the figure also shows that of the packages that do not have defined stability, they always remained close to their initial value. Note that the packages and the number of them between releases vary over time as Figure 2 shows.

As mentioned in Section 3.2, the *category* and *stability* fields are strings without a format. As a consequence, we can find many different categories and stabilities that are not significant to show in the charts (*e.g.,* empty string). These findings are similar to other studies [9, 10], which analyze the current Hackage repository. This is mainly because developers do not follow a protocol to fill these fields.

5 RESULTS: ANSWERING RQS

This section answers the five research questions presented at the beginning of this paper. Each answer is accompanied by a graph or table at least.

5.1 Regarding RQ1

Figure 5 shows a dependency network graph for three releases of Stackage: first, middle, and the last one in 2020. At a first glance, we can clearly observe that the number of dependencies increases with the number of new packages in the repository. During the Stackage evolution, we can observe that the top 8 packages keep their dependency ranks: time, mtl, transformer, text, containers, bytestring, filepath, and vector. Although the mtl package is not the most used (in terms of dependencies), we can mention this package has been widely used in the evolution of the repository. Finally, this figure shows that categories of the top packages are varied.

According to versions of dependencies per package, we present two findings. First, Figure 6a shows the number and percentage of packages whose dependency versions are not available in its Stackage release, meaning that are non-stable dependencies for that release. Although these packages are few, there is a growing trend that might affect the stability promise of Stackage. Second, Figure 6b shows that some packages depend on the same package but with different ranges of versions; making incompatible dependencies of packages.

5.2 Regarding RQ2

To answer this question we use Figures 6c and 6d, which show two box graphs that answer the average of dependencies that are direct and indirect respectively.

Figure 6c shows that the number of direct dependencies is less than 10 and is almost constant around the Stackage evolution. Few packages are outside of this average, where the outlier number of direct dependencies is around 90. Instead, Figure 6d shows that the number of indirect dependencies is higher and its variance is also higher. Regarding indirect dependencies, many packages can be considered outliers, showing that the average (less than 10) of these dependencies is not so representative.

5.3 Regarding RQ3

The first release contains only 800 packages and then many packages have been added to Stackage in the following releases. Figure 6e shows how often these packages are updated between releases. Whereas we can see a higher percentage of packages are updated between releases 2-22 and 6-35, there is also a smaller number of updates between releases 12-14 and 13-19. An explanation for this finding is the interval of time between releases. In the beginning there is an average of 6 months per release, then there is only an average of 3 months per release.

5.4 Regarding RQ4

Figure 6f shows the use of different monads in Stackage. In the figure, we can see that the State and Reader monads have a strong presence against to other monads. This presence may be due to both monads allow developers to express stateful pieces of code. In addition, the figure shows that the uses of Error and Exception are inversely proportional, possibly due to Exception is able to work as a replacement for Error in various scenarios. In fact, Exception increases its presence while Error decreases in each release.

5.5 Regarding RQ5

As this research question is composed of two subquestions, we used two graphs to answer. Figure 6g shows the packages added and removed from releases that import mtl modules. First releases add a higher amount of packages than the last releases; indeed, the last releases remove more packages than added. Between releases 7-24 and 11-24, we can see a considerable fluctuation of packages that are added and removed. As a big picture, this figure shows that from the release 11-22, more packages that import mtl are removed than added. On the contrary, Figure 6h shows the number of packages that start and stop using mtl modules. The first finding is that few packages add/remove the use of mtl over time (approximately 3 packages per release). We explained this result because of the strong need for monads to implement certain operations (*e.g.,* mutable states).

Summary. We can observe the top 8 most used packages, including mtl, remain constant. In addition, the use of mtl in added packages has apparently decreased over time. As downside points, we can highlight two ones. First, a growing trend of packages *a)* are depending on other packages whose versions are not available in a particular release and *b*) have incompatible version dependencies with other packages; opening a potential stability issue. Second, Stackage presents unstructured fields for metadata such as *category* and *stability*, which is an issue that comes from the Hackage repository because Stackage uses its metadata.

6 RELATED WORK

We follow guidelines that come from Mining Software Repository (MSR) [16] to carry out the exploratory study of Stackage. As this article has shown, that through empirical studies carried out on the repository packages, it is possible to know the identity that it has been forging throughout its evolution. This section discusses studies about the evolution of repositories for different programming languages and then Haskell.

In [4], the authors conduct a study of NPM [25], a repository for JavaScript, about its unstable dependencies. The goal study is to analyze the lag in packages that have vulnerable dependencies. By lag in dependencies, the authors refer to the period of time from an error is detected in the dependency to this error is solved. The authors in [19] explain how updating or eliminating dependencies affects a repository that strongly depends on any of them, rising to potential errors in the repository. The study is carried out in the repositories of JavaScript, Ruby, and Rust by making a network of their dependencies to analyze their evolution. Finally in [27], the paper presents an exploratory and descriptive study of packages in the CRAN repository of the R programming language to give a vision of the distribution and the identity that R has for the developer community.

Regarding Haskell, we can find few studies that explore the language community in a known repository. On the one hand, the authors in [9, 10] explore how the developer community uses different kinds of monads in Hackage. On the other hand, the authors in [3] use Hackage to research how developers use generic programming language constructs, tools, and libraries in Haskell. However, none of these studies analyze the evolution of the repositories over time.

7 CONCLUSION AND FUTURE WORK

A repository for a programming language, which allows developers to access and share packages, is a starting point to create a community around a language. Even, a repository is more relevant for the community if it can claim the *stability* feature; as an example, Stackage, which is a curated repository for stable Haskell packages in the Hackage repository. In Stackage, maintainers make available LTS releases that contain a set of packages with specific versions. Although Stackage is widely used (in its industrial target), we are not aware of many empirical studies on how this repository has evolved, including the use of its defining characteristic in Haskell: monads. As a consequence, there is a lack of potential guidelines that Stackage maintainers may follow. To the best of our knowledge, this paper is the first large-scale analysis of the evolution of the Stackage repository: fourteen Long-Term Support (LTS) releases (period 2014 - 2020). As findings, we can highlight the following points. First, there is a growing number of packages that *a*) depend on packages whose versions are not available in a release and *b*) have dependencies with incompatible versions. Second, the `mtl` package is the sixth most used package in Stackage. Third, like previous studies [9, 10], the repository maintainers should be careful when providing unstructured fields for metadata such as *stability* or *category* because developers do not follow a protocol to provide this kind of information. Finally, the use of the `Error` monad seems to be replaced by `Except`, giving a development focus to the Haskell community to work with more packages related to the `Except` monad.

REFERENCES

[1] Algehed, M., and Russo, A. Encoding DCC in Haskell. In *Proceedings of the 2017 Workshop on Programming Languages and Analysis for Security* (Dallas, USA, Oct. 2017), ACM, pp. 77–89.

[2] Benton, N., Hughes, J., and Moggi, E. Monads and Effects. In *Applied Semantics* (Caminha, Portugal, Sept. 2002), G. Barthe, P. Dybjer, L. Pinto, and J. Saraiva, Eds., pp. 42–122.

[3] Bezirgiannis, N., Jeuring, J., and Leather, S. Usage of Generic Programming on Hackage. In *Proceedings of the 9th ACM SIGPLAN workshop on Generic programming - WGP '13* (New York, USA, 2013), p. 47.

[4] Chinthanet, B., Kula, R. G., McIntosh, S., Ishio, T., Ihara, A., and Matsumoto, K. Lags in the release, adoption, and propagation of npm vulnerability fixes. *Empirical Software Engineering 26*, 3 (may 2021), 47.

[5] De Meuter, W. Monads as a theoretical foundation for aop. In *International Workshop on Aspect-Oriented Programming at ECOOP* (Jyväskylä, Finland, June 1997), vol. 25, pp. 31–36.

[6] Decan, A., Mens, T., Claes, M., and Grosjean, P. On the Development and Distribution of R Packages. In *Proceedings of the 2015 European Conference on Software Architecture Workshops* (New York, NY, USA, sep 2015), ACM, pp. 1–6.

[7] Decan, A., Mens, T., and Constantinou, E. On the Evolution of Technical Lag in the NPM Package Dependency Network. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)* (sep 2018), IEEE, pp. 404–414.

[8] Decan, A., Mens, T., and Grosjean, P. An Empirical Comparison of Dependency Network Evolution in Seven Software Packaging Ecosystems. *Empirical Software Engineering 24*, 1 (feb 2019), 381–416.

[9] Figueroa, I. A Preliminary Assessment of how Monads are Used in Haskell. In *Proceedings of the 21st Brazilian Symposium on Programming Languages* (New York, NY, USA, sep 2017), ACM, pp. 1–7.

[10] Figueroa, I., Leger, P., and Fukuda, H. Which Monads Haskell Developers Use: An Exploratory Study. *Science of Computer Programming 201* (2021), 102523.

[11] Gamma, E., Helm, R., Johnson, R., and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software.* Professional Computing Series. Addison-Wesley, October 1994.

[12] Gonzalez-Barahona, J. M., Sherwood, P., Robles, G., and Izquierdo, D. Technical lag in software compilations: Measuring how outdated a software deployment is. In *Open Source Systems: Towards Robust Practices* (Buenos Aires, Argentina, May 2017), Springer International Publishing, pp. 182–192.

[13] Hackage. A central package repository for Haskell, Sept. 2020. http://hackage.haskell.org. Last visited: 01/04/2021.

[14] Haskell. The Haskell Language, Sept. 2020. http://www.haskell.org. Last visited: 01/04/2021.

[15] haskell.org. Haskell Cabal, Apr. 2020. Last visited: 09/04/2021.

[16] Hemmati, H., Nadi, S., Baysal, O., Kononenko, O., Wang, W., Holmes, R., and Godfrey, M. W. The MSR Cookbook: Mining a Decade of Research. In *Proceedings of the 10th Working Conference on Mining Software Repositories* (Piscataway, NJ, USA, 2013), MSR '13, IEEE Press, pp. 343–352.

[17] Jaskelioff, M. Monatron: An Extensible Monad Transformer Library. *Lecture Notes in Computer Science 5836* (2011), 233–248.

[18] Jones, M. P. Functional programming with overloading and higher-order polymorphism. In *Advanced Functional Programming, First International Spring School on Advanced Functional Programming Techniques-Tutorial Text* (Båstad, Sweden, May 1995), pp. 97–136.

[19] Kikas, R., Gousios, G., Dumas, M., and Pfahl, D. Structure and Evolution of Package Dependency Networks. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)* (may 2017), IEEE, pp. 102–112.

[20] Liang, S., Hudak, P., and Jones, M. Monad transformers and modular interpreters. pp. 333–343.

[21] Matplotlib. Matplotlib: A Library for Creating Static, Animated, and Interactive Visualizations in Python, Apr. 2021. http://matplotlib.org. Last visited: 01/04/2021.

[22] Moggi, E. *Computational lambda-calculus and monads.* University of Edinburgh, 1988.

[23] Moggi, E. Notions of computation and monads. *Information and Computation 93*, 1 (1991), 55–92.

[24] Morris, J. G. Experience Report: Using Hackage to Inform Language Design. In *Proceedings of the Third ACM Haskell Symposium on Haskell* (2010), Haskell '10, pp. 61–66.

[25] npm. Build Amazing Things, Apr. 2021. https://www.npmjs.com. Last visited: 08/04/2021.

[26] Pandas. A Fast, Powerful, Flexible and Easy to use Open Source Data Analysis and Manipulation Tool, Apr. 2021. https://pandas.pydata.org. Last visited: 09/04/2021.

[27] Plakidas, K., Schall, D., and Zdun, U. Evolution of the R Software Ecosystem: Metrics, Relationships, and their Impact on Qualities. *Journal of Systems and Software 132* (oct 2017), 119–146.

[28] pypi. Find, Install and Publish Python Packages with the Python Package Index, Apr. 2021. https://pypi.org. Last visited: 08/04/2021.

[29] Scrapy. An Open Source and Collaborative Framework for Extracting the Data you need from Websites, Apr. 2020. https://www.scrapy.org. Last visited: 09/04/2021.

[30] Stackage. Stackage: A Curated Repository for Stable Haskell Packages, Apr. Sept. https://www.stackage.org. Last visited: 01/04/2021.

[31] Tarjan, R. Depth-First Search and Linear Graph Algorithms. *SIAM journal on computing 1*, 2 (1972), 146–160.

[32] Wadler, P. Comprehending monads. In *Proceedings of the 1990 ACM Conference on LISP and Functional Programming* (Nice, France, June 1990), pp. 61–78.

[33] Wadler, P. Monads for functional programming. In *International School on Advanced Functional Programming* (Båstad, Sweden, May 1995), pp. 24–52.